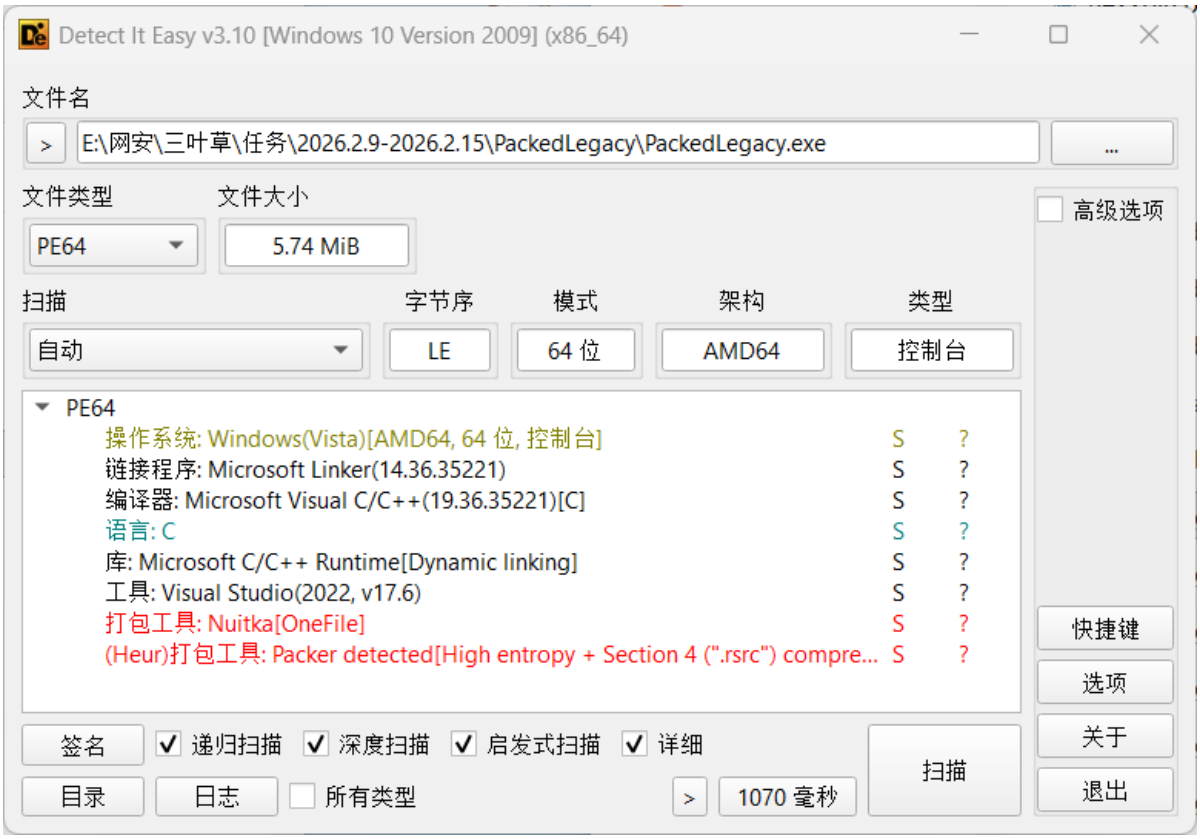
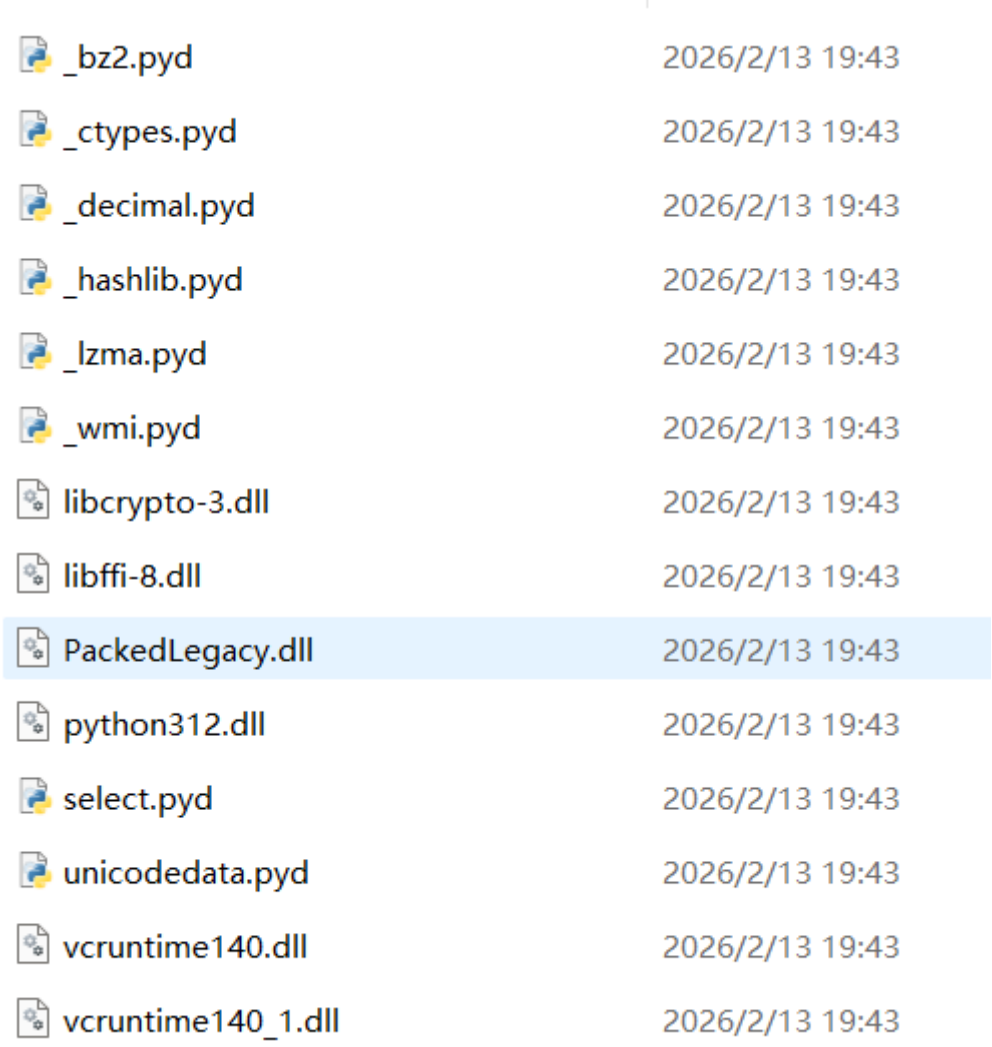
- 用die打开，可以看到是nuikta直接打包单文件

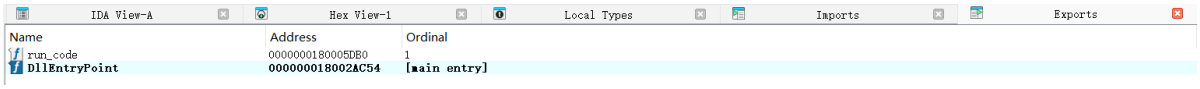


- 使用工具,提取相关文件

```
PS [redacted]\2026.2.9-2026.2.15\PackedLegacy\wp> nuitka-extractor PackedLegacy.exe
[+] Processing PackedLegacy.exe
[+] File type: PE
[+] Payload size: 5931760 bytes
[+] Payload compression: true
[+] Beginning extraction...
[+] Total files: 14
[+] Successfully extracted to PackedLegacy.exe_extracted
```



- 重点关注PackedLegacy.dll文件
- 用ida打开，观察这个动态链接库的导出表，发现只有两个接口



- run_code函数非常可疑，跟进

```
void __noreturn run_code()
{
  sub_180005790();
}
```

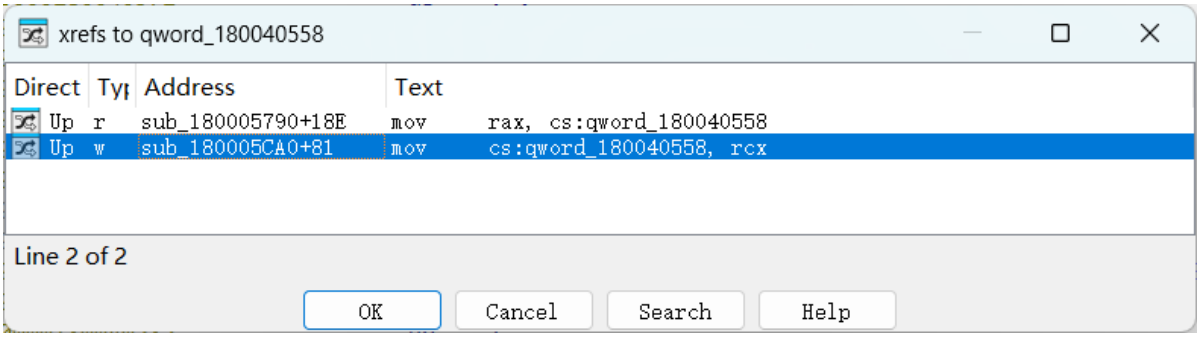
```
// write access to const memory has been detected, the output may be wrong!
void __fastcall __noreturn sub_180005790(unsigned int a1, __int64 *a2)
{
  const wchar_t *v4; // rax
  __int64 v5; // rbx
  __int64 v6; // rax
  unsigned int v7; // eax

  signal(11, Function);
  sub_180005D40();
  sub_180005CA0();
  Py_DebugFlag = 0;
```

```
  Py_InspectFlag = 0;
  Py_InteractiveFlag = Py_InspectFlag;
  Py_OptimizeFlag = 0;
  Py_DontWriteBytecodeFlag = Py_OptimizeFlag;
  Py_NoUserSiteDirectory = 1;
  Py_IgnoreEnvironmentFlag = 0;
  Py_VerboseFlag = Py_IgnoreEnvironmentFlag;
  Py_BytesWarningFlag = 0;
  Py_UTF8Mode = Py_BytesWarningFlag;
  Py_FrozenFlag = 1;
  Py_NoSiteFlag = 1;
  v4 = (const wchar_t *)sub_180025B90("NUITKA_ORIGINAL_ARGV0");
  if ( v4 )
  {
    qword_180040578 = (__int64)wcsdup(v4);
    sub_18002A4A0("NUITKA_ORIGINAL_ARGV0");
  }
  qword_180040578 = *a2;
  qword_180040548 = (__int64)a2;
  Py_SetProgramName();
  dword_180040550 = a1;
  sub_180005DC0(a1, a2);
  sub_1800059B0();
  v5 = PyThreadState_Get();
  Py_NoSiteFlag = 1;
  PySys_SetArgv(a1, qword_180040548);
  sub_1800211B0(v5);
  sub_180001000(v5);
  sub_180001CF0(v5);
  sub_180021610();
  sub_180021B50();
  sub_180021C50();
  sub_180021C00();
  sub_180021F30();
  sub_180021BA0();
  sub_180022390();
  sub_180022470();
  sub_1800295C0();
  sub_1800295A0();
  sub_180005F80(v5);
  sub_180025D00();
  sub_180029DF0(v5);
  PyImport_FrozenModules = qword_180040558;
  sub_1800012D0(v5);
  PyWarnings_Init();
  sub_180029350(v5);
  sub_18002A340(v5, "PATH", qword_180040560);
  sub_18002A340(v5, "PYTHONHOME", qword_180040568);
  v6 = sub_180005770();
  PyDict_DelItemString(v6, "__main__");
  sub_1800055E0(v5);
  sub_180005620(v5, "__main__", 0LL);
  v7 = sub_1800056F0(v5);
  Py_Exit(v7);
  JUMPOUT(0x1800059A9LL);
}
```

- 查询资料可知该函数是**Nuitka 编译生成的可执行程序的核心入口函数**，作用是初始化 Python 运行时环境、配置 Python 全局运行参数、处理命令行参数和 Nuitka 专属环境变量，最终执行编译后的 Python 主程序逻辑并处理程序退出。
- 我们需要重点关注冻结模块PyImport_FrozenModules，它的值来自于qword_180040558

```
.data:0000000180040558 qword_180040558 dq ?                    ; DATA XREF: sub_180005790+18E↑r
.data:0000000180040558                                         ; sub_180005CA0+81↑w
```

- 交叉引用，发现还有一处引用



- 跟进

```
// write access to const memory has been detected, the output may be wrong!
__int64 sub_180005CA0()
{
  _QWORD *v0; // rbx
  __int64 v1; // rbx
  char *v2; // rdi
  size_t v3; // rbx
  __int64 result; // rax

  v0 = (_QWORD *)PyImport_FrozenModules;
  if ( PyImport_FrozenModules )
  {
    if ( *PyImport_FrozenModules )
    {
      do
        v0 += 4;
      while ( *v0 );
    }
    v1 = ((__int64)v0 - PyImport_FrozenModules) >> 5;
  }
  else
  {
    LODWORD(v1) = 0;
  }
  v2 = (char *)malloc(32LL * ((int)v1 + 156));
  v3 = 32LL * (int)v1;
  memcpy(v2, PyImport_FrozenModules, v3);
  result = sub_180001230(&v2[v3]);
  qword_180040558 = PyImport_FrozenModules;
  PyImport_FrozenModules = v2;
  return result;
}
```

- 进入sub_180001230

```
__int64 __fastcall sub_180001230(__int64 a1)
{
  __int64 result; // rax
  int *i; // r9
  __int64 v4; // r8
  int v5; // edx
  int v6; // ecx

  if ( !byte_180040390 )
  {
    sub_180026B90(0LL, qword_18003F970, ".bytecode");
    byte_180040390 = 1;
  }
  result = a1 + 16;
  for ( i = (int *)&unk_18003BADC; ; i += 4 )
  {
    v4 = *(_QWORD *)(i - 3);
    *(_QWORD *)(result - 16) = v4;
    *(_QWORD *)(result - 8) = qword_18003F970[*(i - 1)];
    v5 = *i;
    *(_DWORD *)result = *i;
    *(_DWORD *)(result + 4) = (unsigned int)*i >> 31;
    v6 = -v5;
    *(_QWORD *)(result + 8) = 0LL;
    if ( v5 > 0 )
      v6 = v5;
    *(_DWORD *)result = v6;
    if ( !v4 )
      break;
    result += 32LL;
  }
  return result;
}
```

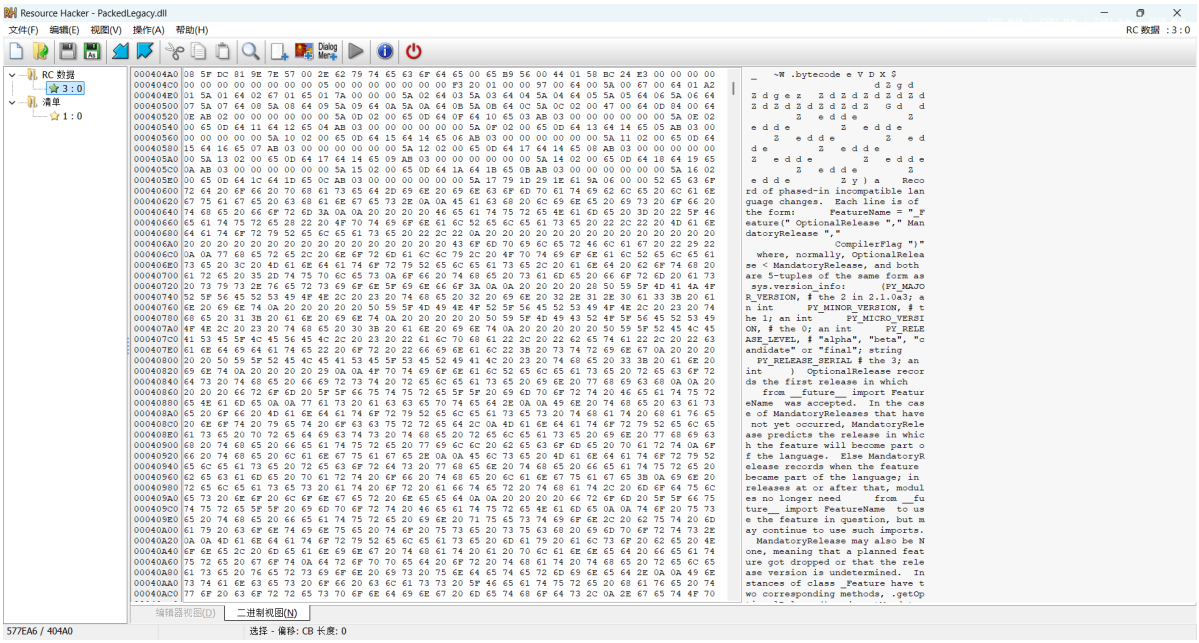
- 进入sub_180026B90

```
__int64 __fastcall sub_180026B90(__int64 a1, __int64 a2, const char *a3)
{
  HMODULE v6; // rbx
  HRSRC ResourceA; // rax
  HGLOBAL Resource; // rax
  char *v9; // rax
  int v10; // ebx
  __int64 v11; // r8
  const char *v12; // rdi
  int v13; // ebx
  size_t v14; // rax
  const char *v15; // rcx
  int v16; // edx
  const char *v17; // rdi
  int v18; // ebx
  size_t v19; // rax
  __int64 result; // rax
```

```
__int64 v21; // rbx

if ( !byte_180040826 )
{
  v6 = hModule;
  if ( !hModule )
  {
    GetModuleHandleExA(6u, (LPCSTR)sub_180025B50, &hModule);
    v6 = hModule;
  }
  ResourceA = FindResourceA(v6, (LPCSTR)3, (LPCSTR)0xA);
  Resource = LoadResource(v6, ResourceA);
  v9 = (char *)LockResource(Resource);
  Str = v9;
  v10 = *(_DWORD *)v9;
  Str = v9 + 4;
  v11 = *((unsigned int *)v9 + 1);
  Str = v9 + 8;
  if ( (unsigned int)sub_180024750(0LL, v9 + 8, v11) != v10 )
  {
    puts("Error, corrupted constants object");
    abort();
  }
  byte_180040826 = 1;
}
if ( strcmp(a3, ".bytecode") && byte_180040825 != 1 )
{
  qword_180041BD0 = PyDict_New();
  qword_180041BD8 = PyDict_New();
  qword_180041BE0 = PyDict_New();
  qword_180041BE8 = PyDict_New();
  qword_180041BF0 = PyDict_New();
  qword_180041BF8 = PyDict_New();
  qword_180041C00 = PyDict_New();
  qword_180041C08 = PyDict_New();
  byte_180040825 = 1;
}
v12 = Str;
v13 = strcmp(a3, Str);
v14 = strlen(v12);
v15 = &v12[v14 + 5];
v16 = *(_DWORD *)&v12[v14 + 1];
if ( v13 )
{
  do
  {
    v17 = &v15[v16];
    v18 = strcmp(a3, v17);
    v19 = strlen(v17);
    v16 = *(_DWORD *)&v17[v19 + 1];
    v15 = &v17[v19 + 5];
  }
  while ( v18 );
}
result = (__int64)(v15 + 2);
if ( *(_WORD *)v15 )
```

```
  {
    v21 = *(unsigned __int16 *)v15;
    do
    {
      result = sub_180022A80(a1, a2, result);
      a2 += 8LL;
      --v21;
    }
    while ( v21 );
  }
  return result;
}
```

- 重点关注　ResourceA = FindResourceA(v6, (LPCSTR)3, (LPCSTR)0xA);，发现从这里获取资源数据
- 使用Resource Hacker提取id为3的资源



- 编写解析脚本

```
import io
import struct

def read_uint32(bio):
    return struct.unpack("<I", bio.read(4))[0]

def read_uint16(bio):
    return struct.unpack("<H", bio.read(2))[0]

def read_utf8(bio):
    bs = b""

    while True:
        bs += bio.read(1)
        if b"\x00" in bs:
            break
    return bs[:-1].decode("utf-8")

def main():
```

```
    with open("main.bin", "rb") as f_in:
        bs = f_in.read()

    bio = io.BytesIO(bs)
    hash_ = read_uint32(bio)
    size = read_uint32(bio)
    print(f"hash: {hex(hash_)}")
    print(f"size: {hex(size)}")

    while bio.tell() < size:
        blob_name = read_utf8(bio)
        blob_size = read_uint32(bio)
        blob_count = read_uint16(bio)
        print(f"name: {blob_name}, size: {hex(blob_size)}, count:
{hex(blob_count)}")
        bio.seek(bio.tell() + (blob_size - 2))

if __name__ == "__main__":
    main()
```

- 运行脚本

```
PS E:[redacted]\2026.2.9-2026.2.15\PackedLegacy\wp> python3 run_code.py
hash: 0x81dc5f08
size: 0x577e9e
name: .bytecode, size: 0x56b965, count: 0x144
name: , size: 0x371, count: 0x6b
name: __main__, size: 0xc1a8, count: 0x2a
```

- 发现存在main，返回ida找到解析函数，再根据解析函数，写出对应的解析脚本

```
unsigned __int8 *__fastcall sub_180022A80(__int64 a1, _QWORD *a2, unsigned __int8
*a3)
{
  unsigned __int8 *v3; // r15
  void **v4; // rbx
  _QWORD *v5; // r14
  unsigned __int8 *v7; // rdi
  __int64 v8; // r12
  __int64 v9; // rax
  __int64 v10; // r15
  __int64 v11; // rbx
  __int64 v12; // rax
  __int64 v13; // rax
  __int64 v14; // rbx
  __int64 v15; // rsi
  __int64 v16; // r12
  __int64 (__fastcall *v17)(); // rcx
  __int64 v18; // rax
  __int64 v19; // r12
  __int64 v20; // rax
  __int64 v21; // rbx
  __int64 v22; // rax
  __int64 v23; // rax
  __int64 v24; // rax
  __int64 v25; // rbx
  __int64 v26; // r13
```

```
unsigned __int64 v27; // rdx
unsigned __int64 v28; // rax
void *v29; // rsp
__int64 v30; // rax
void **p_Src; // rbx
unsigned __int64 v32; // rax
void *v33; // rsp
void *v34; // rsp
void **v35; // r12
__int64 v36; // r14
__int64 v37; // rax
void **v38; // r12
__int64 v39; // rax
__int64 v40; // r12
__int64 v41; // rsi
signed __int64 v42; // r14
__int64 v43; // rax
int v44; // eax
__int64 v45; // r12
__int64 v46; // rax
__int64 v47; // rbx
__int64 v48; // r8
__int64 v49; // rax
_QWORD *v50; // rcx
__int64 *v51; // rax
_QWORD *v52; // rdx
__int64 v53; // r12
__int64 v54; // r15
__int64 v55; // rsi
__int64 v56; // rax
__int64 (__fastcall *v57)(); // rcx
__int64 Item; // rax
__int64 v59; // r13
unsigned __int64 v60; // rax
__int64 v61; // rcx
unsigned __int64 v62; // rcx
void *v63; // rsp
void *v64; // rsp
void **v65; // r15
__int64 v66; // rax
__int64 v67; // r15
__int64 i; // rsi
__int64 v69; // rax
__int64 v70; // rcx
__int64 v71; // rax
__int64 v72; // rdi
__int64 v73; // rbx
__int64 v74; // rax
__int64 v75; // rsi
int v76; // edi
int *v77; // r15
__int64 v78; // r12
__int64 v79; // rsi
__int64 v80; // r9
unsigned __int8 v81; // al
__int64 v82; // rcx
```

```
unsigned __int8 *v83; // r8
unsigned __int8 v84; // dl
int *v85; // rbx
bool v86; // zf
__int64 v87; // rbx
__int64 v88; // rdx
__int64 v89; // rax
__int64 AttrString; // rax
__int64 v91; // rax
__int64 v92; // rax
__int64 v93; // rcx
size_t v94; // rbx
_DWORD *v95; // rdx
int v96; // eax
int v97; // eax
char *v98; // rdi
__int64 v99; // rbx
size_t v100; // rax
__int64 v101; // r9
size_t v102; // rbx
int v103; // eax
char *v104; // rdi
__int64 v105; // rbx
__int64 v106; // rax
__int64 v107; // rax
__int64 v108; // rax
_DWORD *v109; // r12
__int64 v110; // rax
_DWORD *v111; // r15
_DWORD *v112; // rbx
_QWORD *v113; // rsi
_QWORD *v114; // rbx
__int64 v115; // rdx
unsigned __int64 v116; // rax
__int64 v117; // rax
__int64 v118; // rax
__int64 v121; // rax
char *v122; // rdi
__int64 v123; // rax
__int16 v124; // bx
int v125; // r12d
int v126; // r13d
__int64 v127; // r15
unsigned __int8 *v128; // rax
int v129; // r11d
int v130; // eax
int v131; // edx
int v132; // eax
int v133; // ecx
int v134; // eax
int v135; // ecx
int v136; // eax
int v137; // ecx
int v138; // eax
int v139; // ecx
int v140; // r8d
```

```
_QWORD *v141; // rbx
__int64 v142; // rax
unsigned int (__fastcall *v143)(_QWORD); // rax
__int64 v144; // rdx
__int64 v145; // rcx
void *Src; // [rsp+50h] [rbp+0h] BYREF
__int64 v148; // [rsp+58h] [rbp+8h] BYREF
void **v149; // [rsp+60h] [rbp+10h] BYREF
_QWORD v150[2]; // [rsp+68h] [rbp+18h] BYREF
_DWORD *v151; // [rsp+78h] [rbp+28h] BYREF
_DWORD *v152; // [rsp+80h] [rbp+30h] BYREF
_DWORD *v153; // [rsp+88h] [rbp+38h] BYREF

v148 = (__int64)a2;
v3 = a3 + 1;
*a2 = 0LL;
v4 = (void **)*a3;
v5 = a2;
v149 = v4;
v7 = a3 + 1;
Src = a3 + 1;
switch ( (int)v4 )
{
  case '.':
    sub_18001A910("Missing blob values\n");
    abort();
  case ':':
    v106 = sub_180022A80(a1, &v151, v3);
    v107 = sub_180022A80(a1, &v152, v106);
    v108 = sub_180022A80(a1, &v153, v107);
    v109 = v153;
    v7 = (unsigned __int8 *)v108;
    v110 = *(_QWORD *)(a1 + 16);
    v111 = v152;
    v112 = v151;
    v113 = *(_QWORD **)(v110 + 269712);
    if ( v113 )
    {
      *(_QWORD *)(v110 + 269712) = 0LL;
      *v113 = 1LL;
    }
    else
    {
      v113 = (_QWORD *)sub_180001900(PySlice_Type);
    }
    if ( !v109 )
      v109 = (_DWORD *)Py_NoneStruct[0];
    if ( !v112 )
      v112 = (_DWORD *)Py_NoneStruct[0];
    if ( !v111 )
      v111 = (_DWORD *)Py_NoneStruct[0];
    if ( *v109 != -1 )
      ++*v109;
    v113[4] = v109;
    if ( *v112 != -1 )
      ++*v112;
```

```
      v113[2] = v112;
      if ( *v111 != -1 )
        ++*v111;
      v113[3] = v111;
      v114 = v113 - 2;
      v115 = *(_QWORD *)(*(_QWORD *)(PyThreadState_GetCurrent() + 16) + 208LL);
      v116 = *(_QWORD *)(v115 + 8);
      *(_QWORD *)v116 = v113 - 2;
      v114[1] &= 3uLL;
      v114[1] |= v116;
      *v114 = v115;
      *(_QWORD *)(v115 + 8) = v113 - 2;
      *v5 = v113;
      goto LABEL_190;
    case ';':
      v117 = sub_180022A80(a1, &v151, v3);
      v118 = sub_180022A80(a1, &v152, v117);
      v7 = (unsigned __int8 *)sub_180022A80(a1, &v153, v118);
      AttrString = sub_180015C80(a1, v151, v152, v153);
      goto LABEL_189;
    case 'A':
      v123 = sub_180022A80(a1, &v151, v3);
      v7 = (unsigned __int8 *)sub_180022A80(a1, &v152, v123);
      AttrString = Py_GenericAlias(v151, v152);
      goto LABEL_189;
    case 'B':
      v97 = sub_180023B40(&Src);
      v98 = (char *)Src;
      v99 = v97;
      AttrString = PyByteArray_FromStringAndSize(Src, v97);
      v7 = (unsigned __int8 *)&v98[v99];
      goto LABEL_189;
    case 'C':
      v124 = sub_180023B40(&Src);
      Src = (void *)sub_180022A80(a1, v150, Src);
      v125 = sub_180023B40(&Src);
      Src = (void *)sub_180022A80(a1, &v151, Src);
      v126 = sub_180023B40(&Src);
      if ( (v124 & 1) != 0 )
      {
        v7 = (unsigned __int8 *)sub_180022A80(a1, &v149, Src);
        Src = v7;
      }
      else
      {
        v7 = (unsigned __int8 *)Src;
        v149 = (void **)v150[0];
      }
      v127 = 0LL;
      v148 = 0LL;
      if ( (v124 & 2) != 0 )
      {
        v128 = (unsigned __int8 *)sub_180022A80(a1, &v148, v7);
        v127 = v148;
        v7 = v128;
        Src = v128;
```

```
    }
    v129 = 0;
    if ( (v124 & 4) != 0 )
    {
      v130 = sub_180023B40(&Src);
      v7 = (unsigned __int8 *)Src;
      v129 = v130 + 1;
    }
    v131 = 0;
    if ( (v124 & 8) != 0 )
    {
      v132 = sub_180023B40(&Src);
      v7 = (unsigned __int8 *)Src;
      v131 = v132 + 1;
    }
    if ( (v124 & 0x30) == 0x30 )
    {
      v133 = 512;
    }
    else if ( (v124 & 0x20) != 0 )
    {
      v133 = 128;
    }
    else
    {
      v133 = 0;
      if ( (v124 & 0x10) != 0 )
        v133 = 32;
    }
    v134 = v133 + 1;
    if ( (v124 & 0x40) == 0 )
      v134 = v133;
    v135 = v134 + 2;
    if ( (v124 & 0x80u) == 0 )
      v135 = v134;
    v136 = v135 + 4;
    if ( (v124 & 0x100) == 0 )
      v136 = v135;
    v137 = v136 + 8;
    if ( (v124 & 0x200) == 0 )
      v137 = v136;
    v138 = v137 + 0x200000;
    if ( (v124 & 0x400) == 0 )
      v138 = v137;
    v139 = v138 + 0x1000000;
    if ( (v124 & 0x800) == 0 )
      v139 = v138;
    v140 = v139 + 0x400000;
    if ( (v124 & 0x1000) == 0 )
      v140 = v139;
    AttrString = sub_1800276F0(
                   Py_NoneStruct[0],
                   v125 + 1,
                   v140,
                   v150[0],
                   (__int64)v149,
```

```
                    (__int64)v151,
                    v127,
                    v126,
                    v129,
                    v131);
      goto LABEL_189;
    case 'D':
      v24 = sub_180023B40(&Src);
      v25 = (int)v24;
      LODWORD(v26) = v24;
      v150[0] = v24;
      v10 = PyDict_NewPresized((int)v24);
      if ( (int)v26 <= 0 )
      {
        v7 = (unsigned __int8 *)Src;
      }
      else
      {
        v27 = 8 * v25;
        v28 = 8 * v25 + 15;
        if ( v28 <= 8 * v25 )
          v28 = 0xFFFFFFFFFFFFFF0LL;
        v29 = alloca(v28 & 0xFFFFFFFFFFFFFF0uLL);
        v30 = v27 + 15;
        p_Src = &Src;
        if ( v27 + 15 < v27 )
          v30 = 0xFFFFFFFFFFFFFF0LL;
        v32 = v30 & 0xFFFFFFFFFFFFFF0uLL;
        v33 = alloca(v32);
        v7 = (unsigned __int8 *)Src;
        v34 = alloca(v32);
        v26 = (unsigned int)v26;
        v35 = &Src;
        v36 = (unsigned int)v26;
        v149 = &Src;
        do
        {
          v37 = sub_180022A80(a1, v35++, v7);
          v7 = (unsigned __int8 *)v37;
          --v36;
        }
        while ( v36 );
        v38 = v149;
        v5 = (_QWORD *)v148;
        do
        {
          v39 = sub_180022A80(a1, v38++, v7);
          v7 = (unsigned __int8 *)v39;
          --v26;
        }
        while ( v26 );
        v40 = SLODWORD(v150[0]);
        if ( SLODWORD(v150[0]) > 0 )
        {
          v41 = 0LL;
          v42 = (char *)v149 - (char *)&Src;
```

```
          do
          {
            PyDict_SetItem(v10, *p_Src, *(void **)((char *)p_Src + v42));
            ++v41;
            ++p_Src;
          }
          while ( v41 < v40 );
          v5 = (_QWORD *)v148;
        }
      }
      v43 = *(_QWORD *)(v10 + 8);
      v14 = qword_180041BF8;
      v15 = *(_QWORD *)(v43 + 120);
      v16 = *(_QWORD *)(v43 + 200);
      *(_QWORD *)(v43 + 120) = sub_180027C40;
      v17 = sub_180027E80;
      goto LABEL_7;
    case 'E':
      while ( *v7++ )
        ;
      goto LABEL_139;
    case 'F':
      *a2 = Py_FalseStruct;
      goto LABEL_190;
    case 'G':
    case 'g':
      v75 = sub_1800178D0(0LL);
      v76 = sub_180023B40(&Src);
      v77 = (int *)sub_1800178D0(31LL);
      if ( v76 <= 0 )
      {
        v7 = (unsigned __int8 *)Src;
      }
      else
      {
        v78 = (unsigned int)v76;
        v7 = (unsigned __int8 *)Src;
        do
        {
          v79 = PyNumber_InPlaceLshift(v75, v77);
          v80 = 1LL;
          v81 = *v7++;
          v82 = v81 & 0x7F;
          if ( v81 >= 0x80u )
          {
            v83 = v7 + 1;
            do
            {
              v84 = *v7;
              v7 = v83;
              v80 <<= 7;
              ++v83;
              v82 += v80 * (v84 & 0x7F);
            }
            while ( v84 >= 0x80u );
          }
```

```
          v85 = (int *)sub_1800178D0(v82);
          v75 = PyNumber_InPlaceAdd(v79, v85);
          if ( *v85 >= 0 )
          {
            v86 = (*(_QWORD *)v85)-- == 1LL;
            if ( v86 )
              (*(void (__fastcall **)(int *))(*((_QWORD *)v85 + 1) + 48LL))(v85);
          }
          --v78;
        }
        while ( v78 );
        LOBYTE(v4) = (_BYTE)v149;
      }
      if ( *v77 >= 0 )
      {
        v86 = (*(_QWORD *)v77)-- == 1LL;
        if ( v86 )
          (*(void (__fastcall **)(int *))(*((_QWORD *)v77 + 1) + 48LL))(v77);
      }
      if ( (_BYTE)v4 == 71 )
        *(_QWORD *)(v75 + 16) |= 2uLL;
      v87 = qword_180041BD0;
      goto LABEL_78;
    case 'H':
      v7 = (unsigned __int8 *)sub_180022A80(a1, v150, v3);
      AttrString = sub_180015C20(v150[0]);
      goto LABEL_189;
    case 'J':
      v91 = sub_180022A80(a1, &v151, v3);
      v92 = sub_180022A80(a1, &v152, v91);
      v93 = (__int64)v151;
      v7 = (unsigned __int8 *)v92;
      if ( !v151 )
        v93 = qword_18003F480;
      v150[0] = v93;
      v150[1] = v152;
      AttrString = sub_18000EC30(a1, PyComplex_Type, v150);
      goto LABEL_189;
    case 'L':
      LODWORD(v19) = sub_180023B40(&Src);
      v20 = PyList_New((int)v19);
      v7 = (unsigned __int8 *)Src;
      v10 = v20;
      if ( (int)v19 > 0 )
      {
        v21 = *(_QWORD *)(v20 + 24);
        v19 = (unsigned int)v19;
        do
        {
          v22 = sub_180022A80(a1, v21, v7);
          v21 += 8LL;
          v7 = (unsigned __int8 *)v22;
          --v19;
        }
        while ( v19 );
      }
```

```
      v23 = *(_QWORD *)(v10 + 8);
      v14 = qword_180041BF0;
      v15 = *(_QWORD *)(v23 + 120);
      v16 = *(_QWORD *)(v23 + 200);
      *(_QWORD *)(v23 + 120) = sub_1800280D0;
      v17 = sub_180028130;
      goto LABEL_7;
    case 'M':
      v7 = a3 + 2;
      switch ( *v3 )
      {
        case 0u:
          *a2 = Py_NoneStruct[1];
          break;
        case 1u:
          *a2 = PyEllipsis_Type;
          break;
        case 2u:
          *a2 = Py_NotImplementedStruct[1];
          break;
        case 3u:
          *a2 = PyFunction_Type;
          break;
        case 4u:
          *a2 = PyGen_Type;
          break;
        case 5u:
          *a2 = PyCFunction_Type;
          break;
        case 6u:
          *a2 = PyCode_Type;
          break;
        case 7u:
          *a2 = PyModule_Type;
          break;
        case 0xAu:
          *a2 = qword_1800422B8;
          break;
        default:
          sub_18001A680("Missing anon value for %d\n", *v3);
          abort();
      }
      goto LABEL_190;
    case 'O':
      while ( *v7++ )
        ;
LABEL_139:
      AttrString = PyObject_GetAttrString(qword_1800405C0, v3);
      goto LABEL_189;
    case 'P':
    case 'S':
      v44 = sub_180023B40(&Src);
      LODWORD(v45) = v44;
      if ( (_BYTE)v4 == 83 )
      {
        v46 = PySet_New(0LL);
```

```
    }
    else
    {
      if ( !v44 )
      {
        v47 = qword_180041C10;
        if ( !qword_180041C10 )
        {
          v48 = PyRuntime;
          if ( *(_DWORD *)(PyRuntime + 10680LL) != -1 )
          {
            ++*(_DWORD *)(PyRuntime + 10680LL);
            v48 = PyRuntime;
          }
          v47 = sub_180012470(a1, PyFrozenSet_Type, v48 + 10680);
          qword_180041C10 = v47;
        }
        v49 = *(_QWORD *)(v47 + 8);
        v7 = (unsigned __int8 *)Src;
        v50 = (_QWORD *)(v49 + 120);
        v51 = (__int64 *)(v49 + 200);
        v52 = v50;
        goto LABEL_38;
      }
      v46 = PyFrozenSet_New(0LL);
    }
    v47 = v46;
    if ( (int)v45 <= 0 )
    {
      v7 = (unsigned __int8 *)Src;
    }
    else
    {
      v59 = (int)v45;
      v60 = 8LL * (int)v45;
      v61 = v60 + 15;
      if ( v60 + 15 < v60 )
        v61 = 0xFFFFFFFFFFFFFFF0LL;
      v62 = v61 & 0xFFFFFFFFFFFFFFF0uLL;
      v63 = alloca(v62);
      v7 = (unsigned __int8 *)Src;
      v64 = alloca(v62);
      v45 = (unsigned int)v45;
      v150[0] = &Src;
      v65 = &Src;
      do
      {
        v66 = sub_180022A80(a1, v65++, v7);
        v7 = (unsigned __int8 *)v66;
        --v45;
      }
      while ( v45 );
      v67 = v150[0];
      for ( i = 0LL; i < v59; ++i )
        PySet_Add(v47, *(_QWORD *)(v67 + 8 * i));
    }
```

```
      v69 = *(_QWORD *)(v47 + 8);
      v50 = (_QWORD *)(v69 + 120);
      v51 = (__int64 *)(v69 + 200);
      v52 = v50;
      if ( (_BYTE)v149 != 83 )
      {
LABEL_38:
        v53 = *v51;
        v54 = *v50;
        v55 = qword_180041C08;
        *v52 = sub_180028190;
        goto LABEL_39;
      }
      v53 = *v51;
      v54 = *v50;
      v55 = qword_180041C00;
      *v50 = sub_180028190;
LABEL_39:
      v56 = *(_QWORD *)(v47 + 8);
      v57 = sub_180028280;
LABEL_40:
      *(_QWORD *)(v56 + 200) = v57;
      Item = PyDict_GetItem(v55, v47);
      if ( Item )
        v47 = Item;
      else
        PyDict_SetItem(v55, v47, v47);
      *(_QWORD *)(*(_QWORD *)(v47 + 8) + 120LL) = v54;
      *(_QWORD *)(*(_QWORD *)(v47 + 8) + 200LL) = v53;
      *v5 = v47;
LABEL_190:
      v141 = (_QWORD *)*v5;
      v142 = *(_QWORD *)(*v5 + 8LL);
      if ( (*(_DWORD *)(v142 + 168) & 0x4000) != 0 )
      {
        v143 = *(unsigned int (__fastcall **)(_QWORD))(v142 + 328);
        if ( !v143 || v143(*v5) )
        {
          v144 = *(v141 - 2);
          if ( v144 )
          {
            v145 = *(v141 - 1);
            *(_QWORD *)(v145 & 0xFFFFFFFFFFFFFFFCuLL) = v144;
            *(_QWORD *)(v144 + 8) ^= (*(_QWORD *)(v144 + 8) ^ v145) &
0xFFFFFFFFFFFFFFFCuLL;
            *(v141 - 1) &= 1uLL;
            *(v141 - 2) = 0LL;
          }
        }
      }
      *v141 = 0xFFFFFFFFLL;
      return v7;
    case 'Q':
      v7 = a3 + 2;
      if ( *v3 )
      {
```

```
        if ( *v3 == 1 )
        {
          AttrString = PyObject_GetAttrString(qword_1800405C0, "NotImplemented");
        }
        else
        {
          if ( *v3 != 2 )
          {
            sub_18001A680("Missing special value for %d\n", *v3);
            abort();
          }
          AttrString = qword_18003F450;
        }
      }
      else
      {
        AttrString = PyObject_GetAttrString(qword_1800405C0, "Ellipsis");
      }
      goto LABEL_189;
    case 'T':
      LODWORD(v8) = sub_180023B40(&Src);
      v9 = PyTuple_New((int)v8);
      v7 = (unsigned __int8 *)Src;
      v10 = v9;
      if ( (int)v8 > 0 )
      {
        v11 = v9 + 24;
        v8 = (unsigned int)v8;
        do
        {
          v12 = sub_180022A80(a1, v11, v7);
          v11 += 8LL;
          v7 = (unsigned __int8 *)v12;
          --v8;
        }
        while ( v8 );
      }
      v13 = *(_QWORD *)(v10 + 8);
      v14 = qword_180041BE8;
      v15 = *(_QWORD *)(v13 + 120);
      v16 = *(_QWORD *)(v13 + 200);
      *(_QWORD *)(v13 + 120) = sub_180028350;
      v17 = sub_1800283B0;
LABEL_7:
      *(_QWORD *)(*(_QWORD *)(v10 + 8) + 200LL) = v17;
      v18 = PyDict_GetItem(v14, v10);
      if ( v18 )
        v10 = v18;
      else
        PyDict_SetItem(v14, v10, v10);
      *(_QWORD *)(*(_QWORD *)(v10 + 8) + 120LL) = v15;
      *(_QWORD *)(*(_QWORD *)(v10 + 8) + 200LL) = v16;
      *v5 = v10;
      goto LABEL_190;
    case 'X':
      v121 = sub_180023B40(&Src);
```

```
      v122 = (char *)Src;
      *v5 = Src;
      return (unsigned __int8 *)&v122[v121];
    case 'Z':
      v7 = a3 + 2;
      switch ( *v3 )
      {
        case 0u:
          v47 = qword_180041C18;
          if ( qword_180041C18 )
            goto LABEL_155;
          v47 = PyFloat_FromDouble();
          qword_180041C18 = v47;
          v56 = *(_QWORD *)(v47 + 8);
          goto LABEL_84;
        case 1u:
          v47 = qword_180041C20;
          if ( qword_180041C20 )
            goto LABEL_155;
          qword_180041C20 = PyFloat_FromDouble();
          *(double *)(qword_180041C20 + 16) = copysign(*(double *)
(qword_180041C20 + 16), -1.0);
          v47 = qword_180041C20;
          v56 = *(_QWORD *)(qword_180041C20 + 8);
          goto LABEL_84;
        case 2u:
          v47 = qword_180041C28;
          if ( qword_180041C28 )
            goto LABEL_155;
          qword_180041C28 = PyFloat_FromDouble();
          *(double *)(qword_180041C28 + 16) = copysign(*(double *)
(qword_180041C28 + 16), 1.0);
          v47 = qword_180041C28;
          v56 = *(_QWORD *)(qword_180041C28 + 8);
          goto LABEL_84;
        case 3u:
          v47 = qword_180041C30;
          if ( qword_180041C30 )
            goto LABEL_155;
          qword_180041C30 = PyFloat_FromDouble();
          *(double *)(qword_180041C30 + 16) = copysign(*(double *)
(qword_180041C30 + 16), -1.0);
          v47 = qword_180041C30;
          v56 = *(_QWORD *)(qword_180041C30 + 8);
          goto LABEL_84;
        case 4u:
          v47 = qword_180041C38;
          if ( qword_180041C38 )
            goto LABEL_155;
          qword_180041C38 = PyFloat_FromDouble();
          *(double *)(qword_180041C38 + 16) = copysign(*(double *)
(qword_180041C38 + 16), 1.0);
          v47 = qword_180041C38;
          v56 = *(_QWORD *)(qword_180041C38 + 8);
          goto LABEL_84;
        case 5u:
```

```
          v47 = qword_180041C40;
          if ( !qword_180041C40 )
          {
            qword_180041C40 = PyFloat_FromDouble();
            *(double *)(qword_180041C40 + 16) = copysign(*(double *)
(qword_180041C40 + 16), -1.0);
            v47 = qword_180041C40;
          }
LABEL_155:
          v56 = *(_QWORD *)(v47 + 8);
          goto LABEL_84;
        default:
          goto LABEL_198;
      }
    case 'a':
    case 'u':
      v100 = strlen((const char *)v3);
      LOBYTE(v101) = (_DWORD)v149 == 97;
      v102 = v100;
      AttrString = sub_18001FFB0(a1, v3, v100, v101);
      v7 = &v3[v102 + 1];
      goto LABEL_189;
    case 'b':
      v96 = sub_180023B40(&Src);
      v7 = (unsigned __int8 *)Src + v96;
      v75 = sub_180016640(Src, v96);
      v87 = qword_180041BE0;
      v88 = v75;
      goto LABEL_79;
    case 'c':
      v94 = strlen((const char *)v3);
      v75 = sub_180016640(v3, v94);
      v7 = &v3[v94 + 1];
      if ( v94 <= 1 )
        goto LABEL_82;
      v87 = qword_180041BE0;
LABEL_78:
      v88 = v75;
LABEL_79:
      v89 = PyDict_GetItem(v87, v88);
      if ( v89 )
      {
        *v5 = v89;
      }
      else
      {
        PyDict_SetItem(v87, v75, v75);
LABEL_82:
        *v5 = v75;
      }
      goto LABEL_190;
    case 'd':
      v95 = (_DWORD *)(48LL * *v3 + PyRuntime + 10720LL);
      if ( *v95 != -1 )
        ++*v95;
      v7 = a3 + 2;
```

```
      *v5 = v95;
      goto LABEL_190;
    case 'f':
      v7 = a3 + 9;
      v150[0] = *(_QWORD *)v3;
      v47 = PyFloat_FromDouble();
      v56 = *(_QWORD *)(v47 + 8);
LABEL_84:
      v55 = qword_180041BD8;
      v57 = sub_1800280B0;
      v54 = *(_QWORD *)(v56 + 120);
      v53 = *(_QWORD *)(v56 + 200);
      goto LABEL_40;
    case 'j':
      v7 = a3 + 17;
      v148 = *(_QWORD *)v3;
      v150[0] = *(_QWORD *)(a3 + 9);
      AttrString = PyComplex_FromDoubles();
      goto LABEL_189;
    case 'l':
    case 'q':
      v70 = sub_180023B40(&Src);
      if ( (_BYTE)v4 != 108 )
        v70 = (unsigned int)-(int)v70;
      v71 = sub_1800178D0(v70);
      v72 = qword_180041BD0;
      v73 = v71;
      v74 = PyDict_GetItem(qword_180041BD0, v71);
      if ( v74 )
      {
        v7 = (unsigned __int8 *)Src;
        *v5 = v74;
      }
      else
      {
        PyDict_SetItem(v72, v73, v73);
        v7 = (unsigned __int8 *)Src;
        *v5 = v73;
      }
      goto LABEL_190;
    case 'n':
      *a2 = Py_NoneStruct[0];
      goto LABEL_190;
    case 'p':
      *a2 = *(a2 - 1);
      goto LABEL_190;
    case 's':
      v148 = PyUnicode_DecodeUTF8(v3, 0LL, "surrogatepass");
      PyUnicode_InternInPlace(&v148);
      AttrString = v148;
      goto LABEL_189;
    case 't':
      *a2 = Py_TrueStruct;
      goto LABEL_190;
    case 'v':
      v103 = sub_180023B40(&Src);
```

```
      v104 = (char *)Src;
      v105 = v103;
      AttrString = PyUnicode_DecodeUTF8(Src, v103, "surrogatepass");
      v7 = (unsigned __int8 *)&v104[v105];
      goto LABEL_189;
    case 'w':
      v148 = PyUnicode_DecodeUTF8(v3, 1LL, "surrogatepass");
      PyUnicode_InternInPlace(&v148);
      AttrString = v148;
      v7 = v3 + 1;
LABEL_189:
      *v5 = AttrString;
      goto LABEL_190;
    default:
LABEL_198:
      sub_18001A680("Missing decoding for %d\n", (_DWORD)v4);
      abort();
  }
}
```

- 对应的解析脚本

```
import io
import sys
import struct
import math

# -------------------------- 核心读取函数（适配原函数逻辑） --------------------------
def read_uint8(bio):
    """读取1字节无符号整数（原函数的unsigned __int8）"""
    b = bio.read(1)
    if not b:
        raise EOFError("Unexpected EOF when reading uint8")
    return struct.unpack("<B", b)[0]

def read_uint16(bio):
    """读取2字节小端无符号整数"""
    b = bio.read(2)
    if len(b) < 2:
        raise EOFError("Unexpected EOF when reading uint16")
    return struct.unpack("<H", b)[0]

def read_uint32(bio):
    """读取4字节小端无符号整数"""
    b = bio.read(4)
    if len(b) < 4:
        raise EOFError("Unexpected EOF when reading uint32")
    return struct.unpack("<I", b)[0]

def read_leb128(bio):
    """
    适配原函数的无符号LEB128解码（对应sub_180023B40）
    原函数中所有容器长度（列表/字典/集合）均使用此编码
    """
    result = 0
```

```
    shift = 0
    while True:
        byte_val = read_uint8(bio)
        # 取低7位，左移累加
        result |= (byte_val & 0x7F) << shift
        # 最高位为0则结束（原函数的LEB128规则）
        if not (byte_val & 0x80):
            break
        shift += 7
        # 防止溢出（原函数的安全校验）
        if shift > 63:
            raise OverflowError("LEB128 integer too large")
    return result

def read_utf8_style_int(bio):
    """
    适配原函数G/g分支的变长整数解码（UTF-8风格的整数编码）
    对应原函数中：
    v81 = v80 & 0x7F;
    if (v80 >= 0x80u) { 循环读取后续字节，低7位累加 }
    """
    result = 0
    shift = 0
    while True:
        byte_val = read_uint8(bio)
        # 取当前字节的低7位
        result += (byte_val & 0x7F) << shift
        # 最高位为0则结束
        if not (byte_val & 0x80):
            break
        shift += 7
    return result

# ------------------------- 核心解码函数（严格对齐原函数case） ----------------------
----
def decode_blob(bio):
    """
    适配原函数sub_180022A80的核心解码逻辑
    返回：(解码后的对象, 处理后的字节流指针位置)
    """
    # 读取标记字节（原函数v4 = *a3）
    type_byte = read_uint8(bio)
    type_char = chr(type_byte)
    current_pos = bio.tell()

    # ------------------------- 基础类型 -------------------------
    if type_char == 'n':
        # Py_None (原函数case 'n')
        return None, current_pos
    elif type_char == 't':
        # Py_True (原函数case 't')
        return True, current_pos
    elif type_char == 'F':
        # Py_False (原函数case 'F')
        return False, current_pos
    elif type_char in ('a', 'u'):
```

```
        # 字符串/Unicode：strlen后指针到末尾+1（原函数v101 = v99; v7 = &v3[v101 + 1]）
        bs = b""
        while True:
            c = bio.read(1)
            if c == b"\x00" or not c:
                break
            bs += c
        # 原函数要求指针移动到字符串长度+1（跳过末尾的0）
        bio.seek(current_pos + len(bs) + 1)
        return bs.decode("utf-8", errors="surrogatepass"), bio.tell()
    elif type_char == 'w':
        # 单字符字符串：读取1字节（原函数case 'w'）
        c = bio.read(1)
        if not c:
            raise EOFError("Unexpected EOF for 'w' type")
        bio.seek(current_pos + 1)
        return c.decode("utf-8", errors="surrogatepass"), bio.tell()
    elif type_char == 'l' or type_char == 'q':
        # 整数：l=正数，q=负数（原函数case 'l'/'q'）
        value = read_leb128(bio)
        if type_char == 'q':
            value = -value
        return value, bio.tell()
    elif type_char in ('G', 'g'):
        # 特殊变长整数（原函数case 'G'/'g'）
        count = read_leb128(bio)
        total = 0
        for _ in range(count):
            # 读取UTF-8风格的整数段
            num = read_utf8_style_int(bio)
            total = (total << 1) + num
        # G标记需要设置第2位标志（原函数*(v74 + 16) |= 2uLL）
        if type_char == 'G':
            return (total, "G_FLAG"), bio.tell()
        return total, bio.tell()
    # ------------------------- 容器类型 -------------------------
    elif type_char == 'T':
        # 元组：LEB128读长度，递归解码元素（原函数case 'T'）
        sub_count = read_leb128(bio)
        elements = []
        for _ in range(sub_count):
            elem, _ = decode_blob(bio)
            elements.append(elem)
        return tuple(elements), bio.tell()
    elif type_char == 'L':
        # 列表：LEB128读长度，递归解码元素（原函数case 'L'）
        list_count = read_leb128(bio)
        elements = []
        for _ in range(list_count):
            elem, _ = decode_blob(bio)
            elements.append(elem)
        return elements, bio.tell()
    elif type_char == 'D':
        # 字典：LEB128读长度，交替解码key/value（原函数case 'D'）
        dict_count = read_leb128(bio)
        o = {}
```

```
        for _ in range(dict_count):
            key, _ = decode_blob(bio)
            value, _ = decode_blob(bio)
            o[key] = value
        return o, bio.tell()
    elif type_char == 'S' or type_char == 'P':
        # 集合/冻结集合（原函数case 'S'/'P'）
        set_count = read_leb128(bio)
        elements = []
        for _ in range(set_count):
            elem, _ = decode_blob(bio)
            elements.append(elem)
        if type_char == 'S':
            return set(elements), bio.tell()
        else:
            return frozenset(elements), bio.tell()
    # ------------------------- 字节/字符串类型 -------------------------
    elif type_char == 'B':
        # 字节数组：LEB128读长度，读取指定字节（原函数case 'B'）
        length = read_leb128(bio)
        bs = bio.read(length)
        if len(bs) < length:
            raise EOFError(f"Expected {length} bytes for 'B' type, got
{len(bs)}")
        return bytearray(bs), bio.tell()
    elif type_char == 'c':
        # 字节字符串：strlen后指针到末尾+1（原函数case 'c'）
        bs = b""
        while True:
            c = bio.read(1)
            if c == b"\x00" or not c:
                break
            bs += c
        # 原函数v7 = &v3[v93 + 1]
        bio.seek(current_pos + len(bs) + 1)
        return bs, bio.tell()
    elif type_char == 'b':
        # 字节字符串：LEB128读长度（原函数case 'b'）
        length = read_leb128(bio)
        bs = bio.read(length)
        if len(bs) < length:
            raise EOFError(f"Expected {length} bytes for 'b' type, got
{len(bs)}")
        return bs, bio.tell()
    # ------------------------- 浮点数/复数 -------------------------
    elif type_char == 'd':
        # 预定义浮点数索引（原函数case 'd'）
        index = read_uint8(bio)
        # 原函数v94 = (48LL * *v3 + PyRuntime + 10720LL)
        return f"<PREDEFINED_FLOAT_INDEX_{index}>", bio.tell()
    elif type_char == 'f':
        # 浮点数：读取8字节双精度（原函数case 'f'，v7 = a3 + 9）
        float_bytes = bio.read(8)
        if len(float_bytes) < 8:
            raise EOFError("Expected 8 bytes for 'f' type float")
        o = struct.unpack('<d', float_bytes)[0]
```

```
        bio.seek(current_pos + 8)  # 原函数指针移动8字节
        return o, bio.tell()
    elif type_char == 'j':
        # 复数：读取16字节（8+8）（原函数case 'j'，v7 = a3 + 17）
        real_bytes = bio.read(8)
        imag_bytes = bio.read(8)
        if len(real_bytes) < 8 or len(imag_bytes) < 8:
            raise EOFError("Expected 16 bytes for 'j' type complex")
        real = struct.unpack('<d', real_bytes)[0]
        imag = struct.unpack('<d', imag_bytes)[0]
        bio.seek(current_pos + 16)  # 原函数指针移动16字节
        return complex(real, imag), bio.tell()
    elif type_char == 'Z':
        # 预定义特殊浮点数（原函数case 'Z'）
        index = read_uint8(bio)
        # 映射原函数的特殊浮点数（NaN/正负无穷）
        z_map = {
            0: math.nan,
            1: -math.inf,
            2: math.inf,
            3: -math.nan,  # 带符号NaN
            4: math.nan,   # 带符号NaN
            5: -math.inf   # 扩展索引
        }
        o = z_map.get(index, f"<PREDEFINED_DOUBLE_INDEX_{index}>")
        bio.seek(current_pos + 1)
        return o, bio.tell()
    # ------------------------- 特殊对象/操作 -------------------------
    elif type_char == 'M':
        # 匿名对象（原函数case 'M'）
        anon_type = read_uint8(bio)
        anon_map = {
            0: None,  # Py_None
            1: Ellipsis,  # PyEllipsis_Type
            2: NotImplemented,  # Py_NotImplementedStruct
            3: "<PyFunction_Type>",
            4: "<PyGen_Type>",
            5: "<PyCFunction_Type>",
            6: "<PyCode_Type>",
            7: "<PyModule_Type>",
            0x0A: "<qword_1800422B8>"  # 补充原函数的0x0A分支
        }
        o = anon_map.get(anon_type, f"<ANON_TYPE_{anon_type}>")
        bio.seek(current_pos + 1)
        return o, bio.tell()
    elif type_char == 'O':
        # 动态属性获取（原函数case 'O'）
        attr_name = b""
        while True:
            c = bio.read(1)
            if c == b"\x00" or not c:
                break
            attr_name += c
        # 原函数while (*v7++); 指针移动到末尾+1
        bio.seek(current_pos + len(attr_name) + 1)
        return f"<DYNAMIC_ATTR_GET:{attr_name.decode('utf-8')}>", bio.tell()
```

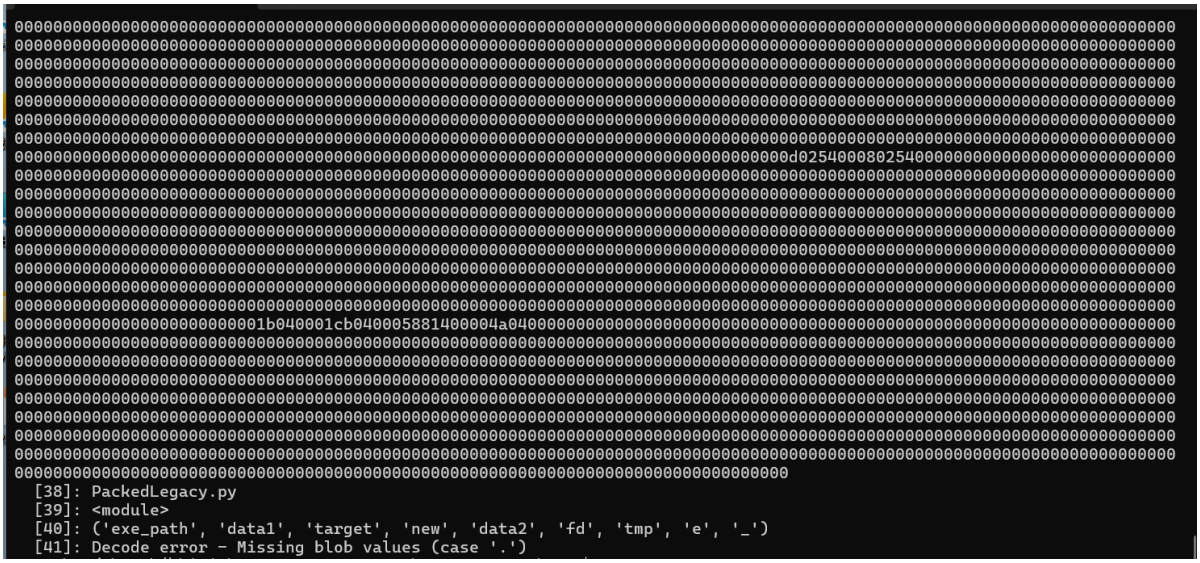
```
    elif type_char == 'Q':
        # 特殊值（原函数case 'Q'）
        special_type = read_uint8(bio)
        special_map = {
            0: Ellipsis,  # Ellipsis
            1: NotImplemented,  # NotImplemented
            2: "<SELF_REFERENCE>"
        }
        o = special_map.get(special_type, f"<SPECIAL_VALUE_{special_type}>")
        bio.seek(current_pos + 1)
        return o, bio.tell()
    elif type_char == 'X':
        # 字节偏移：读取长度后返回指针（原函数case 'X'）
        skip_length = read_leb128(bio)
        # 原函数return &v121[v120]; 移动指针但不创建对象
        bio.seek(current_pos + skip_length)
        return f"<SKIPPED_{skip_length}_BYTES>", bio.tell()
    elif type_char == 'C':
        # 代码对象（原函数case 'C'）
        version = read_leb128(bio)
        argcount = read_leb128(bio)
        flags = read_leb128(bio)
        return f"
<CODE_OBJECT_VERSION_{version}_ARGCOUNT_{argcount}_FLAGS_{flags}>", bio.tell()
    elif type_char == 'A':
        # 泛型别名（原函数case 'A'）
        origin, _ = decode_blob(bio)
        args, _ = decode_blob(bio)
        return f"<GENERIC_ALIAS_ORIGIN_{origin}_ARGS_{args}>", bio.tell()
    elif type_char == ';':
        # Lambda表达式（原函数case ';'）
        code_obj, _ = decode_blob(bio)
        defaults, _ = decode_blob(bio)
        closure, _ = decode_blob(bio)
        return f"<LAMBDA_CODE_{code_obj}_DEFAULTS_{defaults}_CLOSURE_{closure}>",
bio.tell()
    elif type_char == ':':
        # 切片对象（原函数case ':'）
        start, _ = decode_blob(bio)
        stop, _ = decode_blob(bio)
        step, _ = decode_blob(bio)
        return slice(start, stop, step), bio.tell()
    elif type_char == 'p':
        # 堆栈引用（原函数case 'p'）
        return "<STACK_REFERENCE_PREV>", current_pos
    elif type_char == 'v':
        # 变长UTF-8字符串（原函数case 'v'）
        length = read_leb128(bio)
        string_bytes = bio.read(length)
        if len(string_bytes) < length:
            raise EOFError(f"Expected {length} bytes for 'v' type string")
        return string_bytes.decode('utf-8', errors="surrogatepass"), bio.tell()
    elif type_char == 's':
        # 驻留UTF-8字符串（原函数case 's'）
        # 原函数PyUnicode_DecodeUTF8(v3, 0LL, "surrogatepass")
        bs = b""
```

```
        while True:
            c = bio.read(1)
            if c == b"\x00" or not c:
                break
            bs += c
        return bs.decode('utf-8', errors="surrogatepass"), bio.tell()
    elif type_char == 'H':
        # 原函数case 'H'：特殊处理（补充适配）
        sub_obj, _ = decode_blob(bio)
        return f"<SPECIAL_H_TYPE_{sub_obj}>", bio.tell()
    elif type_char == '.':
        # 原函数错误分支：Missing blob values
        raise ValueError("Missing blob values (case '.')")
    else:
        # 原函数默认分支：abort()
        raise ValueError(f"Missing decoding for type 0x{type_byte:02X}
('{type_char}')")

# -------------------------- 主函数（适配原函数的blob解析流程） ----------------------
----
def main():
    if len(sys.argv) < 2:
        print(f"Usage: {sys.argv[0]} <binary_file>")
        sys.exit(1)

    file_path = sys.argv[1]
    with open(file_path, "rb") as f_in:
        bs = f_in.read()
    bio = io.BytesIO(bs)

    # 读取头部（原函数的hash和size）
    try:
        hash_ = read_uint32(bio)
        size = read_uint32(bio)
        print(f"Blob Header - Hash: 0x{hash_:08X}, Size: 0x{size:08X}")
    except EOFError as e:
        print(f"Error reading header: {e}")
        sys.exit(1)

    # 解析blob内容（对齐原函数的循环逻辑）
    while bio.tell() < size:
        # 读取blob名称（以0结尾）
        blob_name = b""
        while True:
            c = bio.read(1)
            if c == b"\x00" or not c:
                break
            blob_name += c
        blob_name = blob_name.decode("utf-8", errors="replace")

        # 读取blob大小和计数
        try:
            blob_size = read_uint32(bio)
            blob_count = read_uint16(bio)
        except EOFError as e:
            print(f"Error reading blob metadata for '{blob_name}': {e}")
```

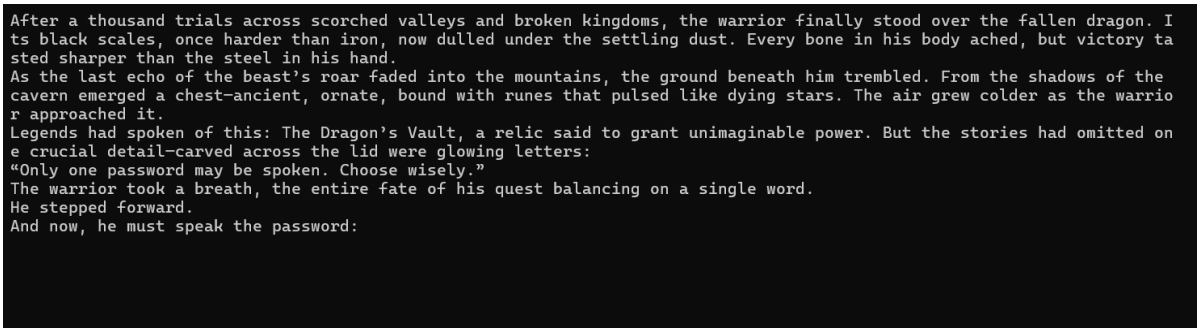
```
            break

        print(f"\nDecoding blob '{blob_name}' (Size: 0x{blob_size:08X}, Count:
{blob_count})...")

        if blob_name == "__main__":
            # 解码__main__ blob的内容
            decoded = []
            for idx in range(blob_count):
                try:
                    obj, new_pos = decode_blob(bio)
                    decoded.append(obj)
                    print(f"  [{idx}]: {obj}")
                except (EOFError, ValueError) as e:
                    print(f"  [{idx}]: Decode error - {e}")
                    break
            break
        else:
            # 跳过其他blob（原函数的指针移动逻辑）
            skip_bytes = blob_size - 2  # 减去已读的blob_count（2字节）
            bio.seek(bio.tell() + skip_bytes)
            print(f"  Skipped (non __main__ blob)")

if __name__ == "__main__":
    main()
```

```
PS E:                                \2026.2.9-2026.2.15\PackedLegacy\wp> python3 jiexi.py main.bin
Blob Header - Hash: 0x81DC5F08, Size: 0x00577E9E

Decoding blob '.bytecode' (Size: 0x0056B965, Count: 324)...
  Skipped (non __main__ blob)

Decoding blob '' (Size: 0x00000371, Count: 107)...
  Skipped (non __main__ blob)

Decoding blob '__main__' (Size: 0x0000C1A8, Count: 42)...
  [0]: binascii
  [1]: unhexlify
  [2]: FILE
  [3]: Error
  [4]: print
  [5]: ('HEX Error',)
  [6]: b'D3"\x11'
  [7]: replace
  [8]: (b'D3"\x11', b'3\x00\x00\x00')
  [9]: tempfile
  [10]: mkstemp
  [11]: ('.exe',)
  [12]: ('suffix',)
  [13]: fdopen
  [14]: wb
  [15]: __enter__
  [16]: __exit__
  [17]: write
  [18]: (None, None, None)
  [19]: subprocess
  [20]: run
  [21]: {'check': False}
  [22]: Runtime Error:
  [23]: uScript Error:
  [24]: exe_path
  [25]: <LAMBDA_CODE_0_DEFAULTS_3_CLOSURE_1>
  [26]: remove
  [27]: time
  [28]: sleep
  [29]: (0.1,)
  [30]: __doc__
  [31]: __file__
  [32]: __cached__
  [33]: __annotations__
  [34]: sys
  [35]: ctypes
  [36]: os
  [37]: 4d5a90000300000004000000ffff0000b8000000000000004000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000800000000e1fba0e00b409cd21b8014ccd2154686973207072 6f6772616d2063616e6e6f742062652072756e20696e20444f53206d6f6465
2e0d0d0a2400000000000000504500004c010800e1e0706900000000000000e0000f030b0102 1c00380000005c000000020000e012000000100000
0050000000004000001000000002000004000000010000000400000000000000c0000000040000b429010003000000000200001000000000100 0
0010000000000001000000000000000000000009000003406000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000004b000001800000000000000000000000000000000000003c910000ec00000000000000
0000000000000000000000000000000000000000000000002e74657874000000c437000000100000003800000040000000000000000000000000600050 60
2e6461746100000b8000000005000000020000003c0000000000000000000000000000400060c02e726461746100007807000000600000000800000
003e0000000000000000000000000000400030402e65685f6672616d080c000000700000000e0000004600000000000000000000000000400030 40
2e627373000000009001000000800000000000000000000000000000000000800060c02e6964617461000034060000090000000800000
0054000000000000000000000000000400030c02e435254000000001800000000a000000020000005c0000000000000000000000000400030c0
2e746c7300000000200000000b000000020000005e0000000000000000000000000400030c0000000000000000000000000000000000000000
```

- 省略中间大段的二进制数据



- 发现中间大段的二进制数据是一个pe文件，提取并转为.exe格式
- 运行发现，确实是程序核心逻辑



- 将该文件拖入ida中，通过字符串定位找到main函数

```
int sub_401DFC()
{
  _DWORD v2[16]; // [esp+1Ch] [ebp-21Ch] BYREF
  _DWORD v3[48]; // [esp+5Ch] [ebp-1DCh] BYREF
  char input[256]; // [esp+11Ch] [ebp-11Ch] BYREF
  int v5; // [esp+21Ch] [ebp-1Ch]
  int i; // [esp+220h] [ebp-18h]
  char v7; // [esp+227h] [ebp-11h]
  void *input_0; // [esp+228h] [ebp-10h]
  size_t Size; // [esp+22Ch] [ebp-Ch]

  sub_402560();
  SetConsoleOutputCP(0xFDE9u);
  puts(
    "After a thousand trials across scorched valleys and broken kingdoms, the
warrior finally stood over the fallen drago"
    "n. Its black scales, once harder than iron, now dulled under the settling
dust. Every bone in his body ached, but vi"
    "ctory tasted sharper than the steel in his hand.");
  puts(aAsTheLastEchoO);
  puts(aLegendsHadSpok);
  puts(&byte_406350);
  puts("The warrior took a breath, the entire fate of his quest balancing on a
single word.");
```

```
  puts("He stepped forward.");
  printf("And now, he must speak the password: ");
  if ( !fgets(input, 256, (FILE *)iob[0]._ptr) )
    return 1;
  Size = strlen(input);
  if ( Size && input[Size - 1] == 10 )
    input[--Size] = 0;
  input_0 = input;
  if ( Size > 2 && input[0] == -17 && input[1] == -69 && input[2] == -65 )
  {
    input_0 = (char *)input_0 + 3;
    Size -= 3;
  }
  sub_401CB1(v3, (int)&unk_405070, dword_405080);
  memset(v2, 0, sizeof(v2));
  memcpy(v2, input_0, Size);
  sub_401D6B(v3, v2, 0x30u);
  v7 = 1;
  for ( i = 0; i <= 47; ++i )
  {
    if ( byte_405040[i] != *((_BYTE *)v2 + i) )
    {
      v7 = 0;
      break;
    }
  }
  if ( v7 )
    puts("The chest glowed open, bathing the warrior in golden light as his
chosen word proved true.");
  else
    puts(
      "The chest vanished forever, and though the warrior remained honored by
all, a quiet doubt lingered—would he ever s"
      "top wondering what might have been?");
  do
    v5 = sub_404748();
  while ( v5 != 10 && v5 != -1 );
  return 0;
}
```